binary_search.cpp

```cpp
#include <stdio.h>
#include <omp.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int n = 10;
    int key = 70;
    int found = 0;
    printf("Array: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\nSearching for: %d\n", key);
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int total_threads = omp_get_num_threads();
        int start = (tid * n) / total_threads;
        int end = ((tid + 1) * n) / total_threads - 1;
        int low = start;
        int high = end;
        while (low <= high && !found)
        {
            int mid = (low + high) / 2;
            if (arr[mid] == key)
            {
                #pragma omp critical
                {
                    if (!found)
                    {
                        found = 1;
                        printf("Element %d found at index %d "
                               "by Thread %d\n",
                               key, mid, tid);
                    }
                }
                break;
            }
            else if (arr[mid] < key)
            {
                low = mid + 1;
            }
            else
            {
                high = mid - 1;
            }
        }
    }
    if (!found)
    {
```

```
52          printf("Element %d not found\n", key);
53      }
54      return 0;
55  }
56
```